

Lab 2 — Extend and constrain your agent

[↑](#) [Retour à « Labs »](#)

Lab 2 — Extend and constrain your agent

4 September · 3h · lab groups of ~15 · follows Lecture 2 Serves: C2, in the same repository as Lab 1.

You produce exactly two artifacts, plus evidence that each one fired. Two, done well and understood, beat six that exist.

Everything you write today is sent on every turn. That is the frame for the whole lab: you are not minimizing cost, you are maximizing quality, and the two point the same way — a rule that earns nothing still crowds out the rules that do. [Killing bloat](#) is the standing payload version of the same argument: put a logging proxy in front of your agent, see what is being sent before you type, and cut what is not earning its tokens. Read it once, whenever; do not stop the lab for it.

Adjacent and outside this course, if you want the *doing* rather than the *managing*: [Beyond vibe coding](#) points at the two sections of Brian Holt's workshop worth your time.

1. Find the rules worth encoding (~30 min)

Look back at Lab 1 and the 3 September slot. Where did the agent do the wrong thing more than once? Where did you correct it in the chat, the same way, twice?

List the recurring corrections. Then route each one:

- Needs **judgement** applied case by case → **instruction**.
- Must **never** happen, no exceptions → **enforcement**.

A third destination exists and is worth a minute of your list, even though it is not one of the two artifacts you are graded on: **a permission entry**. If the correction is *"stop asking me about this command"* or *"never run that one"*, it is neither a document nor a hook — it is an **allow** or a **deny** in your harness's settings. Check the file in rather than leaving it in the git-ignored local variant, or the rule stays on your laptop and is not a rule at all.

Pick one of each. The routing is what is graded, so pick things that genuinely belong where you put them: a rule that only ever has one right answer is a poor choice for instruction, and a rule that needs judgement makes a brittle hook.

2. Write the instruction document (~50 min)

A document the agent **reaches on its own**. Not a prompt you paste.

- Claude Code: a skill — `.claude/skills/<name>/SKILL.md` with **name** and **description** frontmatter — or a section in `CLAUDE.md`.
- Cursor: a rule file. Others: `AGENTS.md`, or your harness's equivalent.

Three things it must have:

- A pointer that fires.** The **description**, or the line naming the doc, is what decides whether the agent reaches it. Lead with what the document is, then name the distinct situations that should trigger it. Cut synonyms.
- The right things at the top.** What the agent *does*, in order, first. Reference material below it. Anything only some paths need, in a separate file behind a pointer.
- A checkable completion criterion.** The agent must be able to tell done from not-done without asking you. "Follows the conventions" is not checkable. "Every new table has a migration and an entry in the schema doc" is.

Do not reach for `/init`. It will offer to read your repository and write a `CLAUDE.md` for you, and the file it writes is the opposite of what step 3b is about: always sent, never reached for, correct on the day and rotting from then on. If you already have one, this is the lab to open it. The argument in full is [Never run `/init`](#).

3. Test that it is reached — this is the lab's real work (~40 min)

Do not assume. Verify.

- Start a **fresh** session.
- Give it a task that *should* trigger the document, worded the way you would naturally word it — not worded to match your pointer.
- Watch. Did it reach the document?

If it did not fire, that is the interesting outcome, not a failure. Sharpen the pointer's wording and try again.

Keep both versions. A student who can show "before, after, and why" is demonstrating exactly what separates Solid from Outstanding, and it costs you nothing but a copy-paste.

Then test the other direction: give it a task that should **not** trigger the document. Does it stay away? Being able to name a case where your pointer misfires is worth more than pretending it never does.

Capture the evidence now: a transcript excerpt or a screenshot showing the document being reached. Reproducing this on 10 September is miserable.

3b. Prune it (~10 min, taken out of step 5)

Your document now works. This step asks whether all of it is doing anything, and it is the cheapest evidence of judgement in the whole lab.

Pick two or three lines you are least sure about. Delete them. Run the same task again in a fresh session.

If the output is the same, the line was a **no-op** — the model was already doing it, and you were paying context load on every request for a sentence that changed nothing. It stays deleted. An instruction can be true, on-topic and well written and still not be worth a token; the only way to find out is to remove it and look, not to argue with yourself about whether it sounds important.

Two other things to hunt for while you are in there:

- A rule whose subject has gone.** Anything guarding against a library, a file or a convention your project no longer has. It does not become harmless when it becomes obsolete — it drags the agent toward a codebase that does not exist. Cut it. Rewriting it as broader advice does not save it; that just produces a no-op with better grammar.
- A rule that exists twice.** If the same convention is in your new document and also in `CLAUDE.md` or a docs page, keep one copy and point at it from the other. Two copies means no authoritative version and two things to update, and saying it twice makes it outweigh the rules you said once.

Write down what you cut and what happened. One line each. *"Deleted 'always write a detailed commit message' — commit messages were identical, so it was a no-op"* is worth more in the oral than any line you added, because it is the difference between a document you assembled and a document you tested.

Where the ten minutes comes from: step 5 is two paragraphs and does not need twenty.

4. Build the enforcement (~40 min)

Something that fires **without the model's cooperation**.

- Claude Code: a `PreToolUse` hook in `.claude/settings.json` that blocks the tool call. `git-guardrails--claude-code` is a working example to read.
- Any harness: a pre-commit hook, or a CI check.

Rules: it must **block**, not warn. It must guard something genuinely non-negotiable in *your* project. And you must be able to answer *why couldn't you just have asked?* — the honest answer is usually "because the agent will comply nine times out of ten, and the tenth costs more than all nine saved".

Be able to say why the tenth exists, because that is the follow-up. It is not that the agent is careless or that the model is weak. It is **non-determinism**: identical prompt, identical context, identical model, different output — the variation is baked into how these systems run and it does not disappear as they get smarter. So "it has never got this wrong" is a sample, not a guarantee, and a rule whose tenth outcome is expensive does not want a good probability. It wants a floor. That sentence, in your own words, is the difference between Basic and Solid on this half of C2.

Capture the evidence: a terminal capture of it blocking something.

5. Write the two-paragraph note (~20 min)

In your repository, a short `docs/agent-control.md` (or wherever your project keeps notes):

- Paragraph 1** — the instruction document: what triggers it, and one case where it should *not*.
- Paragraph 2** — the enforcement: what it blocks, and why instruction was insufficient.

This is what you will read from in the oral. Two paragraphs. Not a report.

Before you leave

- ☐ Instruction document, committed, reached by the agent unprompted
- ☐ Evidence it was reached (transcript excerpt or screenshot), committed
- ☐ Deterministic enforcement, committed
- ☐ Evidence it blocked something, committed
- ☐ `docs/agent-control.md`, two paragraphs
- ☐ Both pointer versions, if you had to sharpen one
- ☐ What you cut in step 3b, and what happened when you did

Between now and 10 September

C1 and C2 repositories freeze at 20:00 on 10 September. Later pushes are not read. Use the time to finish the C1 units of work, not to start something new.